

Later I learnt about Conway's Law, which is about this effect on a larger scale: "Organisations which design systems ... are constrained to produce designs which are copies of the communication structures of these organisations."

We were using concepts such as mocking that are popular today without knowing what they were. With this, I learnt that we can successfully create and use various methods, techniques, or solution approaches in projects without knowing their names.

We started with the simplest thing that worked. For instance, I used a recursive descent parser, because it was the easiest to implement and debug. Using compiler generators would have complicated implementation, and more important, debugging would have been a nightmare. Similarly, we used a simple stack-based interpreter instead of something advanced like Just-in-Time compilation. We used reference counting as the implementation for GC, since that is the simplest to implement. As the first cut, doing the simplest thing that works is the best way to get going. After working in real-world projects, I realised most engineers try complex solutions first and wonder why they fail. I could also relate to Brook's famous second system effect: "The tendency of small, elegant, and successful systems to have elephantine, feature-laden monstrosities as their successors due to inflated expectations."

Requirements play a key role in the success of the project. In our case, we had a well-written and stable specification and that helped complete our implementation.

Without realising it, we had followed the Waterfall model for implementation. I am a fan of iterative methods like Agile, but it is important to know when and where Agile is suitable. Since requirements were known and remained stable, the Waterfall model worked well for this project and was a good choice.

At its core, programming is a fun-filled problem solving activity. The happiness and excitement I experience when I successfully solve a problem is something I find very hard to describe to others. That adrenaline rush when you have a complex project working is something that must be experienced.

We learned many hard lessons as well.

We had spread ourselves too thin by taking on the task of implementing both the compiler and the VM. Functionality completion was not a major problem, but the quality of the code was really bad. Had we taken up only one of these two tasks, we could have achieved both functionality and a quality product.

Good software configuration management practices are essential for a software project. The success of a project or product is not determined solely by the SCM practices, but bad SCM practices will certainly lead to failure.

The quality of the code was poor and this fact was clear

to all of us who wrote the code—we didn't have to run the application to know that the compiler or the VM would crash anytime. So, quality of the code (and of design, by extension) is the most important indicator of quality. (But if we were to start over again, the only way to improve the quality would be to improve the process of developing the code.) Later, when working in large projects, I was quite surprised (and dismayed) to see project managers looking at testing results and defect trends as the main indicator of quality. Testing or defect counts are, of course, some of the indicators of quality; but it is the quality of the code (and design) that is the main determining factor of software quality.

Our effort and time estimate was unrealistic. I later learned about Hofstadter's Law: "It always takes longer than you expect," which is especially applicable for software engineering projects.

We made a costly mistake by thinking that communicating and interacting was a 'waste of time'. Aspects such as effective communication and team work constitute the core of effective software engineering. In fact, I learnt later that our ability to communicate and work as part of a team is more important than our technical skills and abilities, especially when working as part of a large software development team.

I can sum up our experience with this statement: programming is not software engineering. Programming is essentially a personal activity. Programming involves solving a problem through coding and testing. Software engineering is a team activity. Programming is just one aspect of software engineering. Software engineering is about creating a software product that could be used, reused and modified by others; it includes a range of activities—from requirements gathering, modelling or prototyping solutions, to testing, tracking the project's progress and status, applying processes and methods for quality, etc. Software engineering is a structured and disciplined approach towards designing, developing and maintaining high-quality software for the real world. Understanding and applying this distinction between programming and software engineering is the key to successfully developing products or managing projects in the real world. 

References

- [1] The Java Language Specification: <http://docs.oracle.com/javase/6/specs/jls/se7/html/index.html>
- [2] The Java Virtual Machine Specification: <http://docs.oracle.com/javase/6/specs/jvms/se7/html/>
- [3] Inside the Java Virtual Machine, Bill Venners, McGraw-Hill Companies, 2000.

By: Ganesh Samarthayam

The author is a freelance corporate trainer and consultant based in Bengaluru. You can reach him at ganesh.samarthayam@gmail.com.